

# Arhitectura Sistemelor de Calcul

Lect. Dr. Șotropa Diana  
[diana.sotropa@ubbcluj.ro](mailto:diana.sotropa@ubbcluj.ro)



---

Facultatea de Matematică și Informatică  
Universitatea Babeș-Bolyai





# Arhitectura microprocesoarelor X86 (IA-32)

---

# Arhitectura Microprocesoarelor x86 (IA-32)

Cu ce categorii de informație ați lucrat la nivel de limbaj de programare?



# Arhitectura Microprocesoarelor x86 (IA-32)

INSTRUCȚIUNI

DATE (tip de date)

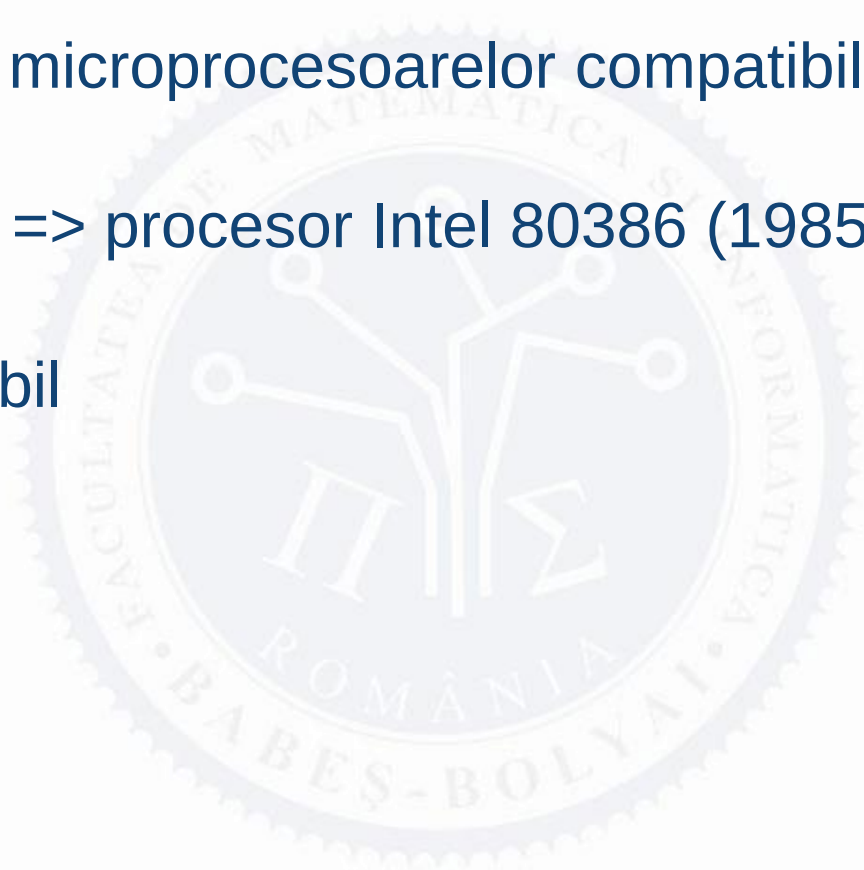
ADRESĂ (calcul de adrese)

# Arhitectura Microprocesoarelor x86 (IA-32)

- Limbaj mașină:
  - Un limbaj numeric înțeles în mod specific de procesorul unui computer (CPU)
- Toate procesoarele x86 înțeleg un limbaj comun al mașinii
- Limbajul mașină – set de numere
- Limbajul de asamblare – instrucțiuni inteligibile
- Fiecare instrucțiune în limbajul de asamblare corespunde unei instrucțiuni unice în limbajul mașină

# Arhitectura Microprocesoarelor x86 (IA-32)

- programarea microprocesoarelor compatibile cu procesoarele Intel IA-32
- procesor x86 => procesor Intel 80386 (1985) => registrii pe 32 de biți
- nu este portabil



# Arhitectura Microprocesoarelor x86 (IA-32)

- Avantaje:
  - libertate de declarare sau transfer de date
  - gamă largă de operatori și expresii de adrese
- Dezavantaje:
  - Un număr mare de detalii necesare a fi stabilite înainte de a scrie efectiv cod semnificativ

# Arhitectura Microprocesoarelor x86 (IA-32)

- Microprocesorul
  - Are viteză
  - Combină comenzi de bază (mai multe instrucțiuni de bază pe care procesorul le poate executa)
  - Orice comandă în orice limbaj de programare este **tradusă** într-un set de instrucțiuni pe care procesorul le poate executa
  - Orice program executabil, indiferent de unde provine, se poate vedea în ce se traduce



Astfel putem vedea limitele limbajului de programare

De ce trebuie să declar variabile?

- Orice operație sau instrucțiune este MAXIM binară

# Structura microprocesorului

Care este unitatea primară de reprezentare a informației?

Care este unitatea primară de acces la informație?

Care este diferența dintre REPREZENTARE și INTERPRETARE?

# Structura microprocesorului

Microprocesor  
x86

EU  
(Executive Unit)

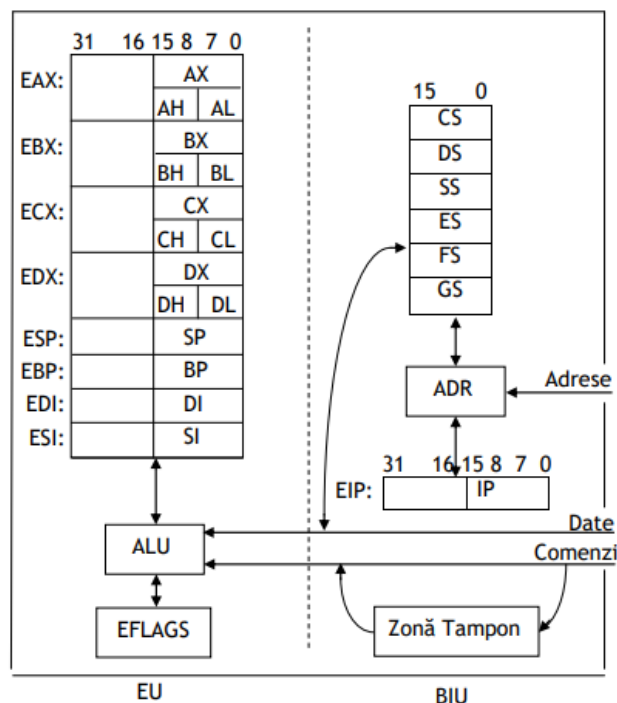
execută instr. mașină prin intermediul  
componentei ALU

BIU  
(Bus Interface Unit)

pregătește execuția fiecărei instrucțiuni mașină

citește o instrucțiune din memorie, o decodifică  
și calculează adresa din memorie a unui  
eventual operand

configurația rezultată este depusă într-o zonă  
tampon cu dimensiunea de 15 octeți, de unde  
va fi preluată de EU

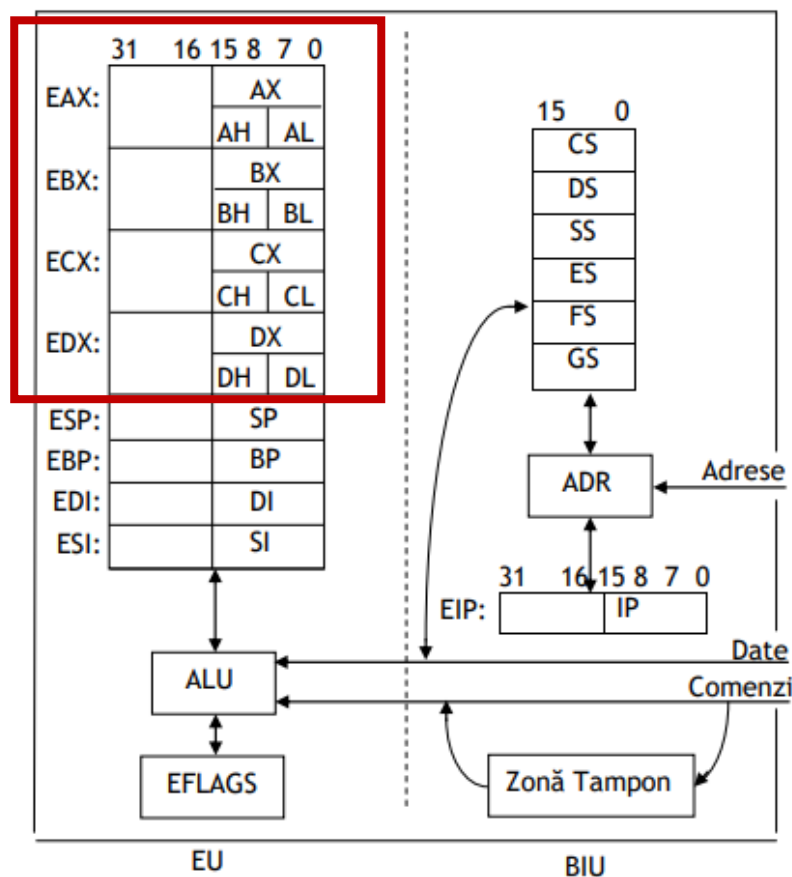


EU și BIU lucrează **în paralel**

În timp ce EU execută instrucțiunea **curentă**,  
BIU pregătește instrucțiunea **următoare**.

Cele două acțiuni sunt **sincronizate** - cea care  
termină prima așteaptă după cealaltă.

# Regiștrii generali EU



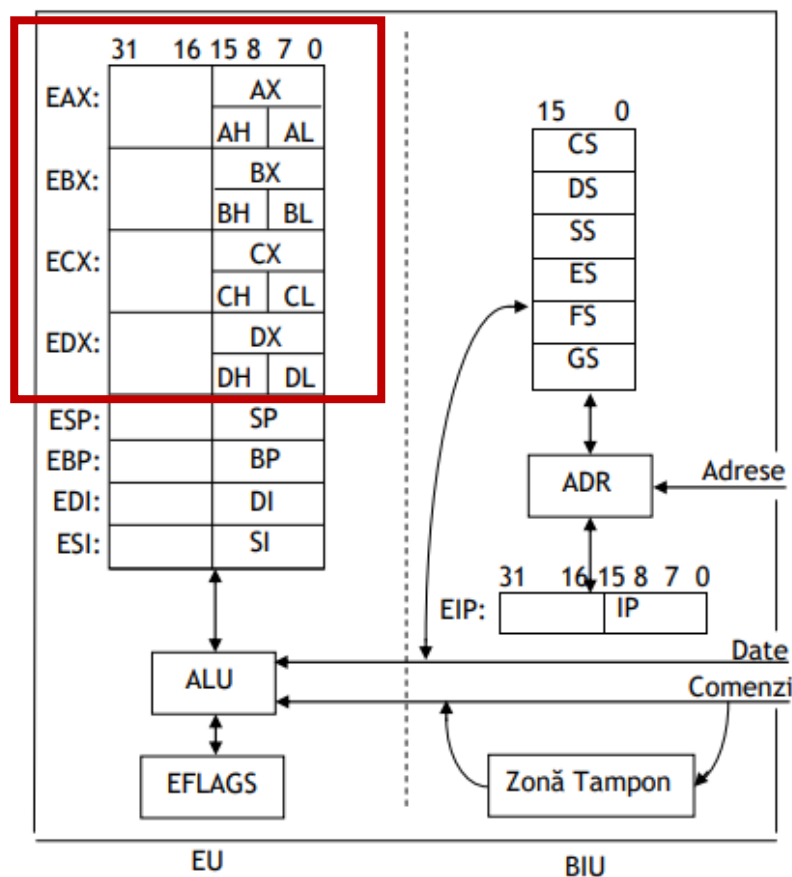
Cum stie procesorul sa distingă între date (variabile / constante), coduri de comenzi sau adrese?

Canale de transmisie (magistrale)

- **A – BUS** – magistrala de adrese
- **C – BUS** – magistrala de comenzi
- **D – BUS** – magistrala de date

Ce înseamnă că un calculator / procesor e pe N biți ?

# Registrii generali EU



- **EAX - registru acumulator**
  - este folosit de către majoritatea instrucțiunilor ca unul dintre operanzi.
- **EBX - registru de bază**
  - provine ca denumire de la *Base Register* folosit în această accepțiune la programarea pe 16 biți
- **ECX - registru contor**
  - pentru instrucțiuni care au nevoie de indicații numerice
- **EDX - registru de date**
  - împreună cu EAX se folosește în calculele ale căror rezultate depășesc un dublucuvânt (32 biți).

# Tipuri de date

- BYTE (8 biți)
- WORD (2 octeți, 16 biți)
- DWORD (4 octeți, 32 biți)
- QWORD (8 octeți, 64 biți)

De ce există QUADWORD?

# De ce există QUADWORD?

- La adunare:

op1 (m cifre) +  
op2 (n cifre)  
-----

suma lor are  **$\max(m,n) + 1$  cifre**

- La înmulțire:

op1 (m cifre) \*  
op2 (n cifre)  
-----

produsul lor are  **$m+n$  cifre**

# Consecințe la nivelul arhitecturii

byte + byte => byte

word + word => word

dword + dword => dword

byte \* byte => word

word \* word => dword

dword \* dword => **qword**

## EDX : EAX (qword)

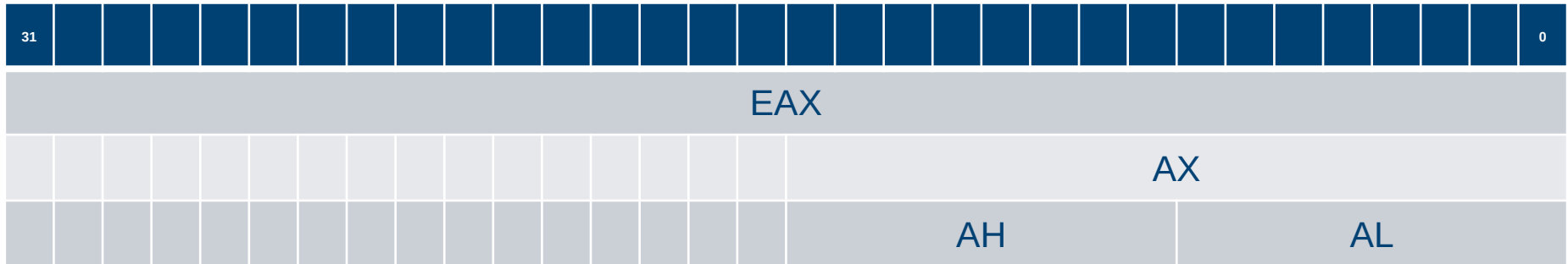
EDX

EAX

Jumătatea superioară (32 biți)

Jumătatea inferioară (32 biți)

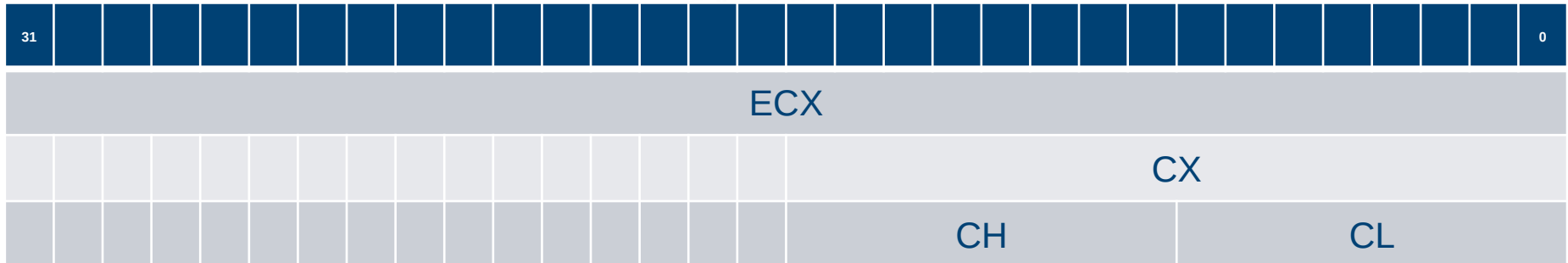
# Regiștrii generali EU



De ce începe numerotarea biților de la 0?

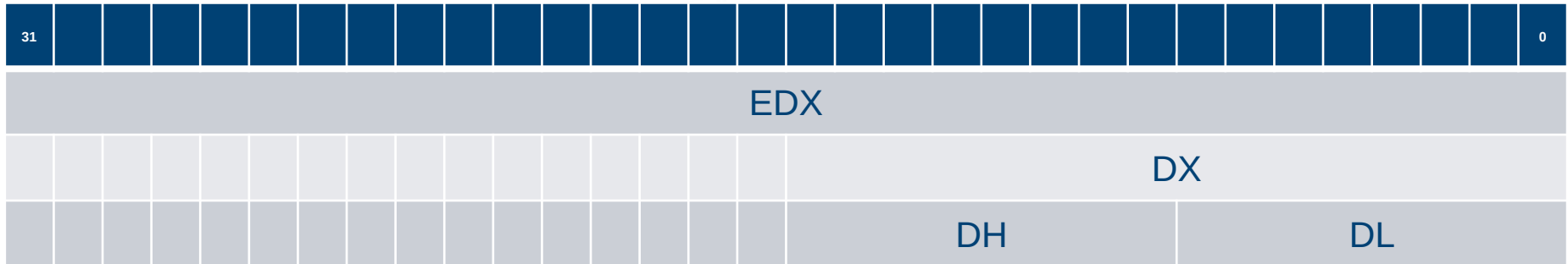
De ce se face numerotarea de la dreapta la stânga?

# Registrii generali EU



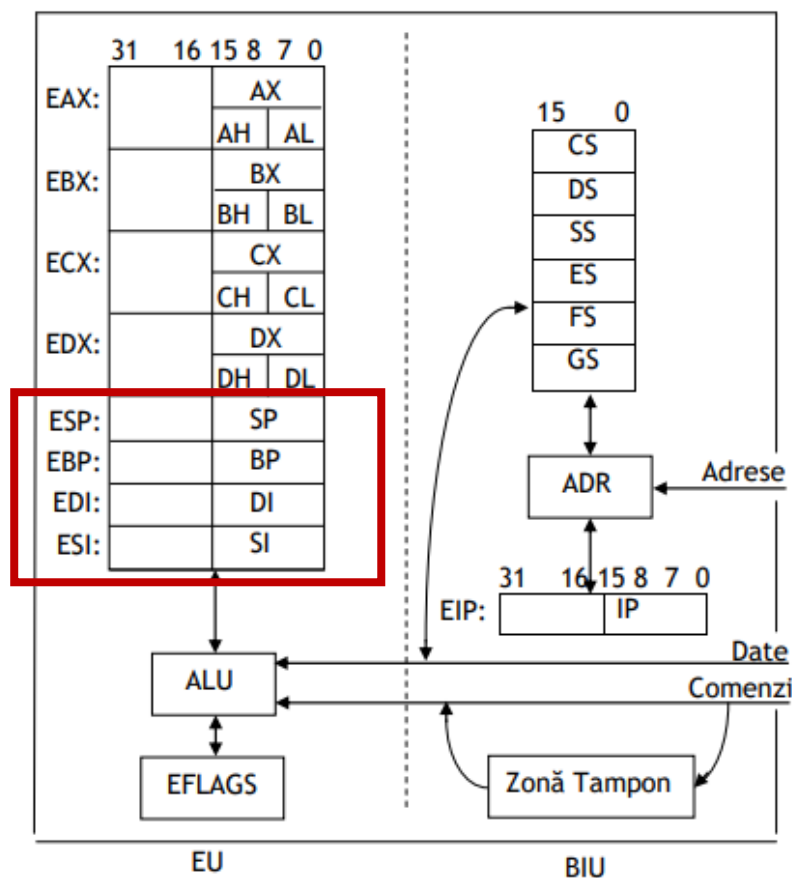
Ce numărăm în cadrul limbajelor de programare?

# Regiștrii generali EU



Cum putem accesa partea superioară a regiștrilor EAX, EBX, ECX EDX?

# Regiștrii generali EU – regiștrii de stivă



- **ESP – registru de stiva**  
(*Stack Pointer*)

- punctează spre elementul ultim introdus în stivă (elementul din vârful stivei).

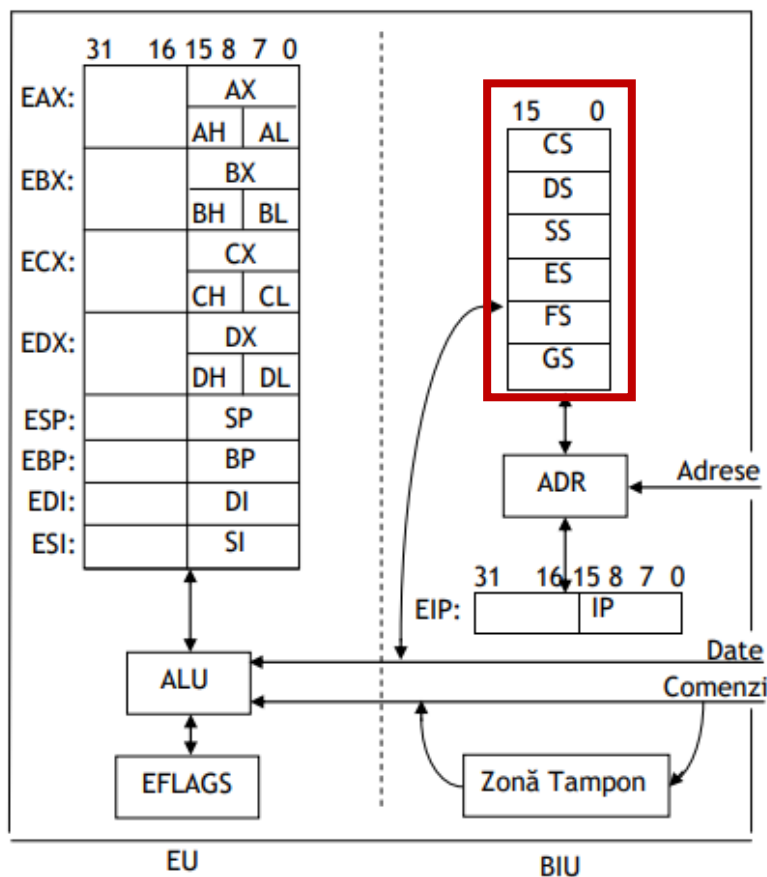
- **EBP – registru de stiva**  
(*Base pointer*)

- punctează spre primul element introdus în stivă (indică baza stivei).

Ce este stiva? Prin ce diferă stiva de coadă?

Ce tipuri de date avem?

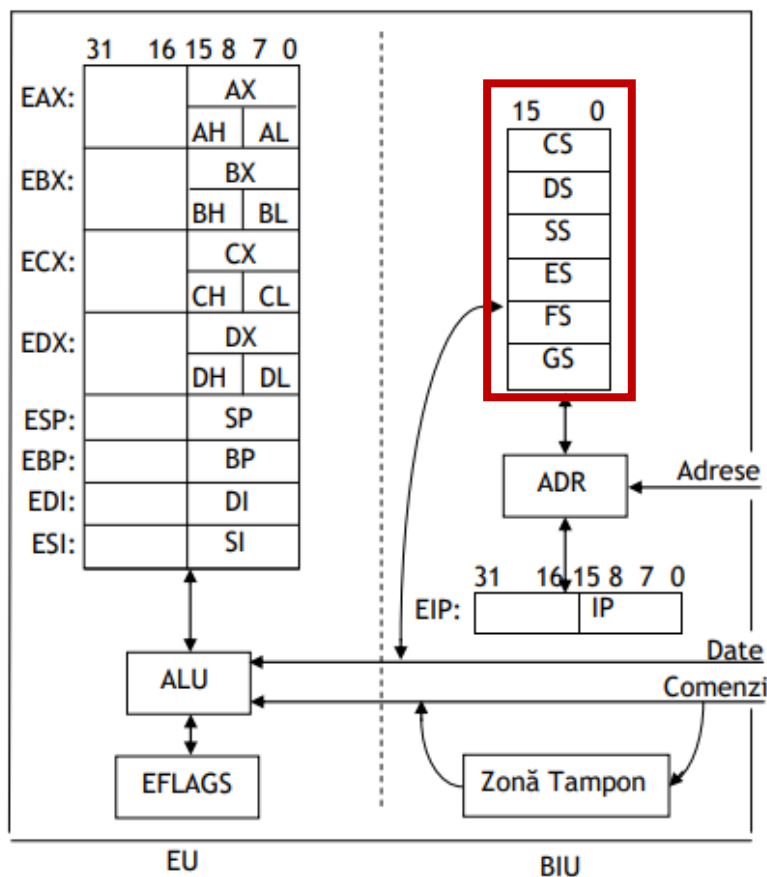
# Regiștrii de segment



- **CS** - Code segment
- **DS** - Data Segment
- **SS** - Stack Segment
- **ES** - Extra Segment
- **FS**
- **GS**

De ce procesorul se ocupă de stivă cu 3 regiștrii și nici unul nu face referire la coadă, heap etc.?

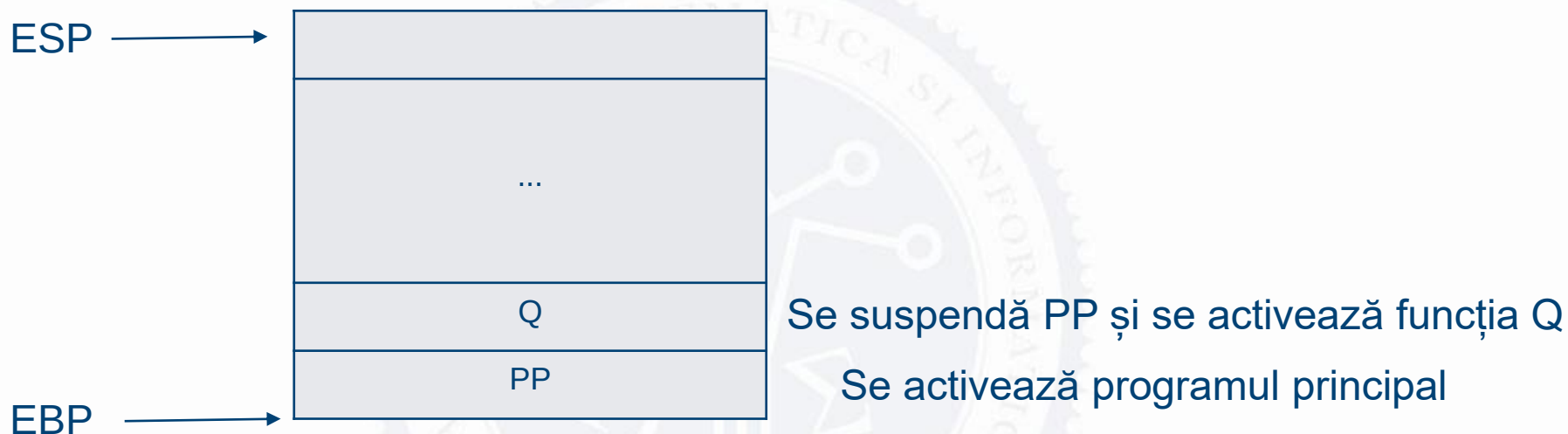
# Regiștrii de segment



- **CS** - Code segment
- **DS** - Data Segment
- **SS** - Stack Segment
- **ES** - Extra Segment
- **FS**
- **GS**

De ce procesorul știe de LIFO și nu e interesat de FIFO?

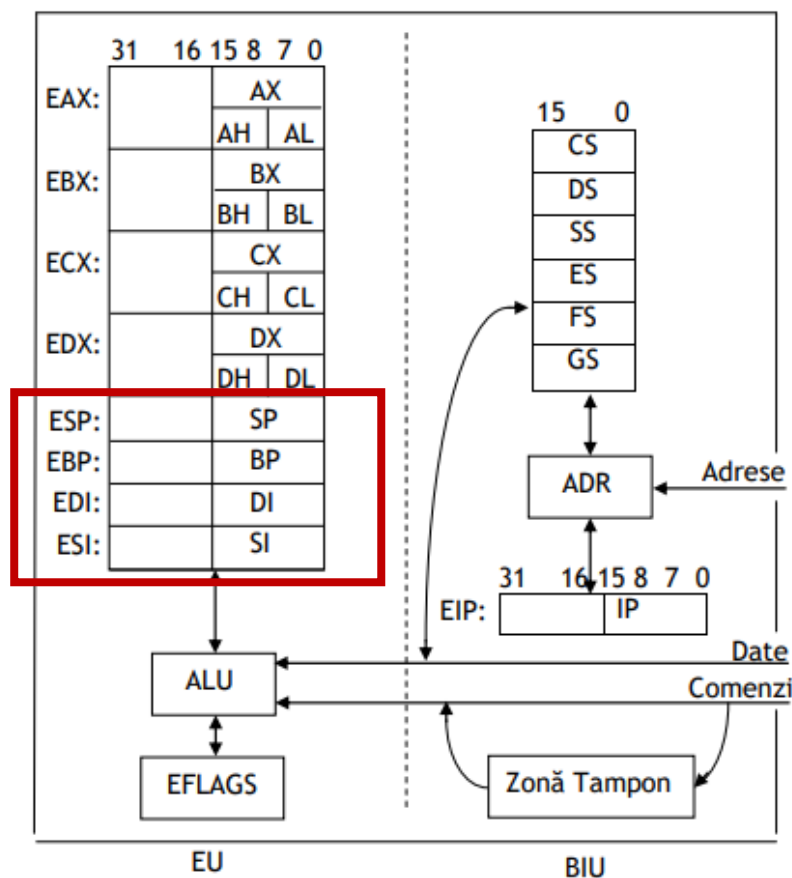
# Stiva de execuție (runtime stack)



Necesar pentru a ști unde a început programul

Rostul limbajelor de asamblare este legătura cu limbajele de programare de nivel înalt (ex. Programare C + ASM)

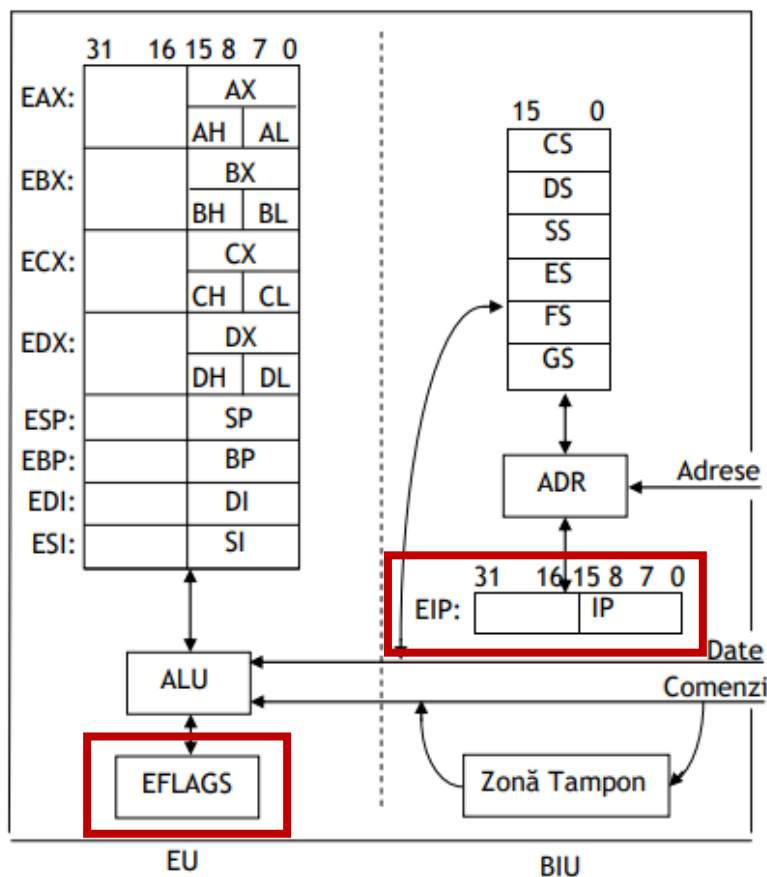
# Regiștrii generali EU – regiștrii de index



- EDI și ESI – **regiștrii de index** (*Destination Index și Source Index*)
  - utilizați de obicei pentru accesarea elementelor din șiruri de octeți sau de cuvinte.

a [5432] – cine e indexul?

# Regiștrii EFLAGS și EIP



- **EFLAGS**

- O configurație a registrului de flaguri indică un rezumat sintetic a execuției fiecărei instrucțiuni.

- **EIP**

- conține adresa următoarei instrucțiuni de executat

# Arhitectura Microprocesoarelor x86 (IA-32)

- Reprezentarea datelor:
  - Cea mai mică unitate de stocare a informației este BIT-ul (0 sau 1)
  - **byte** (octet) = 8 biți
  - **word** (cuvânt) = 2 octeți = 16 biți
  - **doubleword** (dublucuvânt) = 2 cuvinte = 4 octeți = 32 biți
  - **quadword** (tetracuvânt) = 2 dublucuvinte = 4 cuvinte = 8 octeți = 64 biți

# Arhitectura Microprocesoarelor x86 (IA-32)

- Cuvinte rezervate:
  - Numele de instrucțiuni: MOV, ADD, MUL, DIV
  - Numele regiștrilor
  - Numele directivelor
  - Operatori
- Identificatori (variabile, constante, proceduri, etichete):
  - pot contine maxim 247 caractere, litere, cifre și caracterele `_`, `$`, `$$`, `#`, `@`, `~`, `.` și `?`
  - în NASM sunt case sensitive
  - primul caracter trebuie sa fie o litera (A..Z, a..z), `_` și `?`
  - nu poate fi un cuvânt rezervat
- Pentru denumirile implicit parte a limbajului, cum ar fi cuvintele cheie, mnemonicile și numele regiștrilor, nu se diferențiază literele mari de cele mici (acestea sunt case insensitive).

# Arhitectura Microprocesoarelor x86 (IA-32)

- Directive:
  - sunt indicații date asamblorului în scopul generării corecte a octetilor

Ex: relații între modulele obiect, definirea unor segmente, indicații de asamblare condiționată, directive de generare a datelor (*pot defini variabile, macro-uri, proceduri sau pot atribui nume segmentelor de memorie*)

## **Directive pentru definirea variabilelor:**

DB, DW, DD, DQ

## **Directive pentru definirea constantelor:**

EQU

**Exemplu:** zece EQU 10

# Arhitectura Microprocesoarelor x86 (IA-32)

- Etichete:
  - Un identificador care reprezintă adresa unor instrucțiuni sau date

- Etichete de date: identifică locația unei variabile

```
count DW 100  
array DW 1024, 2048  
       4096, 8192
```

- Etichete de cod

```
L1: mov AX, BX  
L2:  
mov AX, BX  
...  
jmp L2
```

# Arhitectura Microprocesoarelor x86 (IA-32)

- Reprezentarea datelor în memorie: little endian
  - O adresă referă o locație de memorie
  - Procesorul x86:
    - permite ca fiecare octet să fie accesibil prin adresă
    - stochează și furnizează datele din memorie folosind reprezentarea little endian
      - Cel mai puțin semnificativ octet este stocat în memorie la ce mai mica adresă alocată
      - Octeții rămași sunt stocați în memorie în octeții următori

## Exemplu:

|   |    |                   |                                                   |
|---|----|-------------------|---------------------------------------------------|
| a | DB | 12h               | ;   12h                                           |
| b | DW | 3456h             | ;   56h   34h                                     |
| c | DD | 7890ABCDh         | ;   CDh   Abh   90h   78h                         |
| d | DQ | 1122334455667788h | ;   88h   77h   66h   55h   44h   33h   22h   11h |

# Arhitectura Microprocesoarelor x86 (IA-32)

OllyDbg - datasegm.exe [CPU - main thread, module datasegm]

File View Debug Trace Plugins Options Windows Help

LE MW TC CR ... K BM H

| Address  | Hex dump          | Assembly                         | Registers (FPU)                        |
|----------|-------------------|----------------------------------|----------------------------------------|
| 00402000 | 6A 00             | PUSH 0                           | EAX 0019FFCC                           |
| 00402002 | FF 15 3C 30 40 00 | CALL DWORD PTR DS:[&msvcrt.exit] | ECX 00402000 datasegm.<ModuleEntryPoin |
| 00402008 | 00 00             | ADD BYTE PTR DS:[EAX],AL         | EDX 00402000 datasegm.<ModuleEntryPoin |
| 0040200A | 00 00             | ADD BYTE PTR DS:[EAX],AL         | EBX 0028B000                           |
| 0040200C | 00 00             | ADD BYTE PTR DS:[EAX],AL         | ESP 0019FF74                           |
| 0040200E | 00 00             | ADD BYTE PTR DS:[EAX],AL         | EBP 0019FF80                           |
| 00402010 | 00 00             | ADD BYTE PTR DS:[EAX],AL         | ESI 00402000 datasegm.<ModuleEntryPoin |
| 00402012 | 00 00             | ADD BYTE PTR DS:[EAX],AL         | EDI 00402000 datasegm.<ModuleEntryPoin |
| 00402014 | 00 00             | ADD BYTE PTR DS:[EAX],AL         | EIP 00402000 datasegm.<ModuleEntryPoin |
| 00402016 | 00 00             | ADD BYTE PTR DS:[EAX],AL         | C 0 ES 002B 32bit 0(FFFFFFFF)          |
| 00402018 | 00 00             | ADD BYTE PTR DS:[EAX],AL         | P 1 CS 0023 32bit 0(FFFFFFFF)          |
| 0040201A | 00 00             | ADD BYTE PTR DS:[EAX],AL         | A 0 SS 002B 32bit 0(FFFFFFFF)          |
| 0040201C | 00 00             | ADD BYTE PTR DS:[EAX],AL         | Z 1 DS 002B 32bit 0(FFFFFFFF)          |
| 0040201E | 00 00             | ADD BYTE PTR DS:[EAX],AL         | S 0 FS 0053 32bit 28E000(FFF)          |
|          |                   |                                  | T 0 CS 002B 32bit 0(FFFFFFFF)          |

Stack [0019FF70]=0  
Imm=0

| Address  | Hex dump    | Disassembly                       |
|----------|-------------|-----------------------------------|
| 0019FF74 | 77 A2 F9 89 | RETURN to KERNEL32.BaseThreadIn   |
| 0019FF78 | 00 28 B0 00 | paaw KERNEL32.BaseThreadInitThunk |
| 0019FF7C | 77 A2 F9 70 | paaw                              |
| 0019FF80 | 00 19 FF DC | ma la                             |
| 0019FF84 | 77 BA 74 B4 | ta w RETURN to ntdll.77BA74B4     |
| 0019FF88 | 00 28 B0 00 | aa (a                             |
| 0019FF8C | 37 6D F9 F9 | aa 7                              |
| 0019FF90 | 00 00 00 00 | aaaa                              |
| 0019FF94 | 00 00 00 00 | aaaa                              |
| 0019FF98 | 00 28 B0 00 | aa (a                             |
| 0019FF9C | 00 00 00 00 | aaaa                              |

Data representation in memory

byte (in data segment) a db 12h  
word (in data segment) b dw 3456h  
doubleword (in data segment) c dd 7890ABCDh  
quadword (in data segment) d dq 1122334455667788h

# Arhitectura Microprocesoarelor x86 (IA-32)

- Instrucțiuni:

- Sunt traduse de către asamblor în limbaj mașină și apoi sunt încărcate și executate de către CPU la runtime
- Formatul unei linii sursă

**[eticheta[:]] [prefixe] [mnemonică] [operanzi] [;comentariu]**

```
NUME_INSTRUCTIUNE destinatie, sursa
```

destinatie = primul operand

sursa = al doilea operand

## Alte formate de instrucțiuni:

```
NUME_INSTRUCTIUNE operand1, operand2
```

```
NUME_INSTRUCTIUNE operand
```

```
NUME_INSTRUCTIUNE
```

# Arhitectura Microprocesoarelor x86 (IA-32)

- Un program în limbaj de asamblare este format din segmente logice:
  - Segmentul de cod: instrucțiuni
  - Segmentul de date: variabile
  - Stiva: parametrii procedurilor, variabile locale, adrese de revenire
- Instrumente necesare:
  - **Notepad++**: scrierea fișierului sursă în limbaj de asamblare (.asm)
  - **NASM**: asamblor, adică programul care citește fișierul sursă și produce fișierul obiect (.obj)
  - **ALINK**: link-editor, adică programul care citește mai multe fișiere obiect și produce fișierul executabil (.exe)
  - **OllyDbg**: debugger

# Abrevieri uzuale

| Abreviere          | Semnificatie                                                                                                                                                                   |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| reg                | Un registru pe 8, 16 sau 32 de biti<br>AH, AL, BH, BL, CH, CL, DH, DL, AX, BX, CX, DX, SI, DI, BP, SP, EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP                                  |
| reg8, reg16, reg32 | Un registru identificat prin numarul de biti<br>reg8: AH, AL, BH, BL, CH, CL, DH, DL<br>reg16: AX, BX, CX, DX, SI, DI, BP, SP<br>reg32: EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP |
| mem                | Un operand din memorie (o variabila)                                                                                                                                           |
| mem8, mem16, mem32 | Un operand din memorie identificat prin numarul de biti                                                                                                                        |
| con                | Un operand imediat (constanta)                                                                                                                                                 |
| con8, con16, con32 | Un operand imediat identificat prin numarul de biti                                                                                                                            |



## FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ UNIVERSITATEA BABEȘ-BOLYAI

Str. Mihail Kogălniceanu nr. 1  
Cluj-Napoca, Cluj, România

[www.cs.ubbcluj.ro](http://www.cs.ubbcluj.ro)